



SMART-CONTRACT SECURITY ASSESSMENT

AEGENT V2 Smart-Contract Threat Model and Security Assessment

Threat Model / Adversary-Oriented Profile

Author and issuer: AEGENT Security Review Team

Assessment date: 2026-07-29

Target environment: BNB Chain mainnet deployment candidate

Assessment type: Point-in-time source review

FINAL DISPOSITION remediation required before mainnet

This report is authored and issued by the AEGENT Security Review Team. It is not authored, reviewed, endorsed, signed, certified, or issued by SlowMist, CertiK, OpenZeppelin, or another independent auditor. No third-party logo, signature, report identifier, certificate, or issuance claim appears in this document. The threat-model structure is project-owned.

Contents

1. Executive assessment

2. Snapshot identity and scope boundary

3. Security objectives

4. Assets and adverse outcomes

5. Actors and capabilities

6. Trust boundaries

TB-01 — User wallet to purchase/redemption endpoints

TB-02 — Registry to execution endpoints

TB-03 — Oracle feeds to Swap

TB-04 — Token contracts to AEGENT accounting

TB-05 — Swap to Vault and Redemption

TB-06 — Owner to privileged state

TB-07 — Vault to immutable beneficiary

TB-08 — Events to off-chain accounting

7. Critical data flows

7.1 Native BNB purchase

7.2 USDT/USDC purchase

7.3 Presale cost-basis refund

7.4 Post-launch redemption

7.5 Purchased-AGNT claim and withdrawals

8. Adversarial scenario register

9. STRIDE-oriented review

10. Accepted findings

MED-01 — Manual-review threshold can be bypassed by splitting redemptions

MED-02 — Purchases can increase refund liability without enforced reserve collateral

MED-03 — Freshness-only oracle validation lacks bounds and deviation circuit breakers

LOW-01 — Fixed UTC daily buckets permit near-two-times throughput at a boundary

INFO-01 — Privileged controls do not enforce a multisig or general timelock

INFO-02 — Immutable recipients and narrow recovery paths create asset-liveness risk

INFO-03 — Settlement events omit configuration evidence needed for complete reconstruction

11. Positive-control analysis

12. Verification evidence

Dynamic tests

Coverage

Static analysis

Toolchain

13. Remediation and retest strategy

14. Limitations

15. Conclusion

1. Executive assessment

The AEGENT V2 contracts form a Registry-controlled purchase and redemption system:

- buyers exchange BNB, USDT, or USDC for AGNT under a scheduled phase and market mode;
- purchase proceeds move to a time-locked vault;
- wallet purchase receipts create cost-basis refund and AGNT-claim accounting;
- redemptions operate either as presale cost-basis refunds or post-launch AGNT-for-USDT settlement;
- owner controls govern modes, windows, delayed post-launch rates, reserves, and eligible withdrawals.

The assessment modeled malicious buyers, malicious redeemers, stale or anomalous oracle values, privileged-key compromise, hostile or nonstandard tokens, rejecting beneficiaries, replayed calldata, stale market configuration, underfunded reserves, and incomplete settlement evidence.

No Critical or High-severity issue was identified in the reviewed bytes. Seven findings remain open:

Severity	Open findings
Critical	0
High	0
Medium	3
Low	1
Informational	3

The Medium findings allow review-threshold splitting, purchase-created refund undercollateralization, and economically anomalous but technically fresh oracle pricing. These prevent a mainnet deployment recommendation.

2. Snapshot identity and scope boundary

The frozen source under [source-snapshot/](#) is the sole controlling scope. Project commit [cae20adbf0497c46906e36cb98bcd3d19232c98b](#) is workspace context only; it is not claimed to contain this uncommitted snapshot.

Frozen source	SHA-256
contracts/AegentMarketRegistry.sol	1e0fc93396402b19d085bbfc25d03be53540e847039dd768c6d47bf379f49aa9
contracts/AegentSaleProceedsVault.sol	983af93be4bb1fcbe2809c9fb01baea34a7bae0ddf12caf05933c5450842cc34
contracts/AegentSwapV2.sol	2b53552a367b757cc31721d31cb2e889dc9391afc293a4cac0b2590332fb515f
contracts/AegentRedemptionV2.sol	2424be9e3141b055d0a27a7b12ab6fc3918d758ddbedc9201ebc454dd467fcc6

Frozen source	SHA-256
contracts/interfaces/IAgentMarketRegistry.sol	fb7b9a35bef353c3c0a5e882da595294b9dad8d81cedada1207bbfe7bb02309
contracts/interfaces/IAgentPurchaseReceiptSource.sol	52abd846c87734040bf375177186bff6d8b56f9b23ddbeeb8a7a4f4c3d5badb
contracts/interfaces/IAgentRedemptionEndpoint.sol	92620d9cc1fec26c42b4d4d293447d4faf9d83aab05a065d7484eff299d89eab
contracts/interfaces/IAgentRegistryBound.sol	bf53c7606cbc5853a74710c9fa31dfd256a31283eedaf7ede01e213b0f12bc84
contracts/interfaces/IAgentSalesVault.sol	eb1005d38941efc7dfd9e8c961eb5a4df10f9a43f630330495dd82b73854c974
contracts/interfaces/IAggregatorV3.sol	2dc0d69856b9cf3579d866c1da3bb9baff2fe505d9702001fcddc47e319a2b7e

Supporting tests and deployment-script tests were executed but are not production source scope. Mocks, frontend/API code, finality databases, wallet code, runtime configuration, deployed bytecode, constructor values, live ownership, reserve balances, and key custody were not assessed.

3. Security objectives

The system should preserve the following properties:

1. **Purchase integrity:** only the active Registry phase/mode/window and current configuration can settle a purchase.
2. **Pricing integrity:** AGNT output must use valid oracle data and the current authorized phase rate.
3. **Slippage and expiry:** a transaction must not settle below user minimum output or after its bounded deadline.
4. **Asset custody:** external calls must not enable reentrant theft or unaccounted token movement.
5. **Receipt integrity:** purchase cost and AGNT entitlement must be bound to the correct wallet and updated atomically.
6. **Refund solvency:** accepted refund liabilities should remain fully collateralized.
7. **Redemption integrity:** presale refunds must not exceed cost basis; post-launch redemptions must collect AGNT.
8. **Replay resistance:** a valid signed transaction payload must not settle twice.
9. **Limit integrity:** wallet/global limits and manual-review controls must retain their intended effect under transaction splitting and boundary timing.
10. **Privilege containment:** owner actions should be delayed, distributed, monitored, and incapable of silently violating protected liabilities.

11. **Forensic completeness:** emitted evidence should identify the governing configuration for each settlement.

4. Assets and adverse outcomes

Asset or state	Required protection	Representative adverse outcome
AGNT purchase inventory	No excess distribution; preserve claims	Oracle overvaluation or accounting error releases too much AGNT
Escrowed purchased AGNT	Preserve all unclaimed wallet entitlement	Owner withdrawal or inconsistent receipt accounting removes claim backing
USDT redemption reserve	Preserve refund/redemption liabilities	Accepted purchases create liabilities above funded reserve
BNB/USDT/USDC sale proceeds	No premature or unauthorized release	Reentrancy, wrong beneficiary, or bypassed sale-end check
Wallet purchase receipts	No cross-wallet attribution, overwrite, or double consumption	Attacker claims or refunds another wallet's entitlement
Redemption execution nonce	Strict monotonic one-use semantics	Replayed calldata settles a second redemption
Registry phase/mode/version	One authoritative execution state	Stale quote executes under changed price or market mode
Oracle observation	Authentic, fresh, and economically plausible	Fresh anomalous value creates invalid quote
Daily/manual-review counters	Preserve real risk-policy intent	Split transactions or midnight boundary evade effective limits
Settlement events	Durable reconstruction	Off-chain accounting cannot prove governing configuration
Owner authority	Minimize unilateral compromise impact	Compromised key changes modes or withdraws eligible funds

5. Actors and capabilities

Actor	Assumed capability	Security posture
Untrusted buyer	Chooses payment asset, amount, minimum output, deadline, and transaction ordering	Fully adversarial
Untrusted redeemer	Splits transactions, times boundaries, replays payloads, and selects permitted parameters	Fully adversarial
MEV searcher	Observes mempool transactions and controls ordering within miner/validator constraints	Fully adversarial
Compromised owner	Executes any owner-only action permitted by code	Modeled as a trust failure

Actor	Assumed capability	Security posture
Faulty oracle	Returns stale, anomalous, delayed, or incomplete values	Modeled
Nonstandard ERC-20	Charges fees, returns unusual values, reenters, or changes balances unexpectedly	Partly modeled; balance checks and SafeERC20 help
Rejecting beneficiary	Refuses native BNB	Modeled as availability failure
Off-chain client/indexer	Supplies stale expected state or reconstructs events incompletely	Untrusted for contract safety
External deployer/operations team	Selects addresses, roles, reserves, and initial state	Out of source scope; must be independently verified

6. Trust boundaries

TB-01 — User wallet to purchase/redemption endpoints

Untrusted calldata crosses into [AagentSwapV2](#) and [AagentRedemptionV2](#). Contract checks must independently enforce amount, phase, mode, version, deadline, minimum output, entitlement, reserve, and nonce. Frontend validation is not trusted.

TB-02 — Registry to execution endpoints

The Registry is the authoritative market-state boundary. Swap and Redemption must reject endpoint mismatch and stale expected configuration. A compromised Registry owner remains a governance risk.

TB-03 — Oracle feeds to Swap

External price data crosses from Chainlink-compatible feeds into BNB and stablecoin valuation. Technical freshness and round validity are necessary but do not guarantee economic plausibility.

TB-04 — Token contracts to AEGENT accounting

AGNT, USDT, and USDC can have external behavior. [SafeERC20](#) and before/after balance checks reduce return-value and fee-on-transfer ambiguity. Token upgradeability and administrative behavior were not assessed.

TB-05 — Swap to Vault and Redemption

Purchase consideration is forwarded to the vault, while AGNT and receipt state interact with Redemption. Atomicity and endpoint binding are required to prevent partial settlement.

TB-06 — Owner to privileged state

The owner can change operational controls and withdraw eligible assets. Two-step ownership and a delayed rate help, but no mandatory multisig/general timelock constrains this boundary.

TB-07 — Vault to immutable beneficiary

Native BNB release invokes an immutable external beneficiary. Rejection cannot redirect theft, but it can block liveness.

TB-08 — Events to off-chain accounting

Indexers and finality systems consume emitted events. Missing configuration-version evidence requires historical state reconstruction and weakens standalone auditability.

7. Critical data flows

7.1 Native BNB purchase

1. Buyer submits expected phase, mode, window, configuration version, minimum AGNT output, and deadline.
2. Swap reads `purchaseTerms()` from the Registry.
3. Swap reads and validates the BNB oracle observation.
4. Swap calculates USD value and AGNT output.
5. Swap transfers or escrows AGNT and records wallet purchase receipt data.
6. Native BNB is forwarded to the sale-proceeds vault.
7. Settlement event is emitted.

Threat focus: stale configuration, stale/anomalous oracle, reentrancy, partial accounting, excess output, wrong vault, event incompleteness.

7.2 USDT/USDC purchase

1. Buyer authorizes the exact stablecoin amount off-chain.
2. Swap verifies current Registry terms.
3. Swap calculates stablecoin USD value using its price feed and decimals.
4. Stablecoin transfer is reconciled by balance delta.
5. AGNT output, receipts, and vault transfer settle atomically.

Threat focus: fee-on-transfer tokens, decimal mismatch, stablecoin depeg, stale-but-acceptable observation, refund-reserve creation.

7.3 Presale cost-basis refund

1. Redeemer submits expected phase, mode, configuration version, minimum USDT output, deadline, and execution nonce.
2. Redemption derives refund value from the wallet's recorded purchase cost basis.

3. Entitlement, reserve, manual-review threshold, wallet/global daily totals, and nonce are checked.
4. Receipt entitlement and reserve/accounting are updated.
5. USDT is transferred and the nonce is consumed atomically.

Threat focus: cross-wallet refund, double refund, splitting around review, UTC boundary burst, insufficient reserve, replay.

7.4 Post-launch redemption

1. Redeemer submits the expected delayed-rate version and bounded parameters.
2. Redemption validates market mode/configuration and rate version.
3. AGNT is collected and reconciled by balance delta.
4. USDT reserve is checked and paid.
5. Counters and nonce are consumed atomically.

Threat focus: stale rate/configuration, missing AGNT transfer, replay, reserve drain, split limits.

7.5 Purchased-AGNT claim and withdrawals

At sale end, a wallet can claim recorded purchased AGNT. Owner withdrawals calculate only balances above protected liabilities and unclaimed entitlements.

Threat focus: double claim, claim/withdraw race, arithmetic truncation, owner removal of protected assets.

8. Adversarial scenario register

Threat ID	Scenario	Existing controls	Residual result
T-01	Replay the same redemption calldata	Per-wallet expected nonce; nonce consumed in successful atomic execution	Mitigated
T-02	Execute a stale quote after phase/mode/configuration changes	Expected phase, mode, window/config version, rate version, deadline, and minimum output	Mitigated
T-03	Reenter during token/native settlement	nonReentrant , checks-effects-interactions, fixed external targets, SafeERC20	Mitigated in reviewed paths
T-04	Use fee-on-transfer behavior to corrupt accounting	Before/after token balance reconciliation	Mitigated for modeled transfer deltas
T-05	Refund more than wallet cost basis	Wallet receipt entitlement and cost-basis decrement	Mitigated
T-06	Claim purchased AGNT twice	Claim state is cleared before transfer	Mitigated

Threat ID	Scenario	Existing controls	Residual result
T-07	Withdraw protected reserve or claim inventory	Eligible withdrawal calculations preserve recorded liabilities	Mitigated in source
T-08	Split redemption to avoid manual review	Review threshold applies only to each transaction	MED-01
T-09	Accept purchases while refund reserve is underfunded	Redemption fails closed, but purchase admission has no projected reserve check	MED-02
T-10	Supply an anomalous but technically fresh oracle answer	Positive answer, timestamp, round-order, and max-age checks only	MED-03
T-11	Burst almost twice a daily limit around UTC midnight	Fixed <code>block.timestamp / 1 days</code> counters	LOW-01
T-12	Compromise a single owner account	Two-step ownership; delayed post-launch rate only	INFO-01
T-13	Block vault release or strand forced/unrelated assets	Immutable recipient and narrow token support	INFO-02
T-14	Dispute which configuration governed a settlement	Some event fields exist, but versions/mode evidence is incomplete	INFO-03

9. STRIDE-oriented review

Category	Modeled examples	Result
Spoofing	Cross-wallet purchase/refund attribution; wrong endpoint binding	Wallet state and Registry binding materially reduce risk
Tampering	Stale phase/mode/version; changed rate; modified minimum output	On-chain comparisons reject stale expected state
Repudiation	Incomplete settlement configuration evidence	Residual INFO-03
Information disclosure	Public contract state and events	No confidential on-chain asset was expected
Denial of service	Underfunded reserve, rejecting beneficiary, stale oracle, UTC limits	MED-02, INFO-02, plus intended fail-closed behavior
Elevation of privilege	Owner compromise or unilateral control	Residual INFO-01
Economic manipulation	Oracle anomaly, review splitting, boundary bursts	MED-01, MED-03, LOW-01

10. Accepted findings

MED-01 — Manual-review threshold can be bypassed by splitting redemptions

Status: Open; no remediation retest

Affected code: [`AagentRedemptionV2.sol:L190-L227`](#), [`AagentRedemptionV2.sol:L694-L731`](#)

Test evidence: [`AagentRedemptionV2.test.cjs:L454-L494`](#)

`manualReviewThresholdUsdt` is compared with only the current transaction's `grossUsdt`. Cumulative wallet usage is checked against a separate daily limit. The supplied test demonstrates that one 201-USDT-equivalent redemption reverts, two separate 200-USDT-equivalent redemptions succeed, and a third is rejected only because the wallet daily amount would exceed 400 USDT.

Threat path: Split one reviewable redemption into multiple sub-threshold calls within the same permitted daily total.

Impact: The intended manual-review/AML control can be avoided while other entitlement, reserve, AGNT, and daily-limit controls remain intact.

Recommendation: Apply the review decision to cumulative rolling/daily wallet value, or require a reviewer-signed authorization bound to wallet, maximum amount, nonce, and expiry. Add negative tests for sequential splitting.

MED-02 — Purchases can increase refund liability without enforced reserve collateral

Status: Open; no remediation retest

Affected code: [`AagentMarketRegistry.sol:L186-L227`](#), [`AagentSwapV2.sol:L249-L289`](#),
[`AagentSwapV2.sol:L358-L380`](#), [`AagentRedemptionV2.sol:L522-L553`](#),
[`AagentSaleProceedsVault.sol:L69-L100`](#)

Test evidence: [`AagentRedemptionV2.test.cjs:L240-L274`](#), [`AagentV2Integration.test.cjs:L98-L107`](#)

`purchaseTerms()` validates schedule, mode, windows, and endpoint compatibility but not reserve collateral. Both native and stable purchase paths can accept consideration and increase wallet refund liability without checking projected USDT reserve coverage. Redemption later fails closed if the reserve is inadequate. The proceeds vault is separate and locked until sale end, so accepted consideration does not automatically fund the redemption reserve.

Threat path: Keep purchase mode open while reserve funding is absent or falls behind new liabilities; buyers receive recorded claims but cannot exercise cost-basis refunds until an operator tops up USDT.

Impact: Bounded insolvency/availability gap for refunds; the system can accept obligations that are not immediately serviceable.

Recommendation: Require projected reserve coverage during each purchase or pre-lock the maximum possible liability before enabling sales. Test atomic rejection for underfunded BNB, USDT, and USDC purchases.

MED-03 — Freshness-only oracle validation lacks bounds and deviation circuit breakers

Status: Open; no remediation retest

Affected code: [`AagentSwapV2.sol:L165-L167`](#), [`AagentSwapV2.sol:L552-L618`](#)

Test evidence: [`AagentSwapV2.test.cjs:L424-L468`](#)

The adapter checks positive answer, nonzero/nonfuture timestamp, completed round ordering, and one global maximum age bounded at six hours. It does not apply per-feed heartbeat, absolute bounds, deviation from prior observations, secondary-source agreement, or automatic pause.

Stablecoin upside is capped at one dollar and a fresh 0.80 price is handled, but a fast depeg may remain hidden inside the allowed age. A fresh anomalously high BNB value can release excess AGNT up to inventory and sale caps.

Threat path: Exploit a feed malfunction or economically stale-within-age observation before the configured age expires.

Impact: Invalid quotes can over-distribute AGNT or produce materially poor user execution.

Recommendation: Use feed-specific heartbeat, absolute min/max, deviation threshold, stablecoin floor, and automatic pause. Add extreme-fresh-value and fast-deviation tests.

LOW-01 — Fixed UTC daily buckets permit near-two-times throughput at a boundary

Status: Open; no remediation retest

Affected code: [`AagentRedemptionV2.sol:L713-L731`](#)

Counters reset at `block.timestamp / 1 days`. Full use immediately before and after a UTC boundary creates almost twice the nominal daily throughput in a short interval.

Impact: Short-window risk exposure exceeds the intuitive daily limit, although entitlement and reserve checks still apply.

Recommendation: Use a rolling 24-hour or token-bucket design, or explicitly size and document the worst-case two-epoch burst.

INFO-01 — Privileged controls do not enforce a multisig or general timelock

Status: Open; no remediation retest

Affected code: [`AagentMarketRegistry.sol:L96-L162`](#), [`AagentSwapV2.sol:L339-L355`](#), [`AagentRedemptionV2.sol:L426-L516`](#)

Two-step ownership and delayed post-launch rate activation are positive controls. A single owner can still control broader market and eligible-withdrawal actions without an enforced multisig/general timelock.

Recommendation: Use a documented multisig owner, separate emergency and treasury roles, timelock material changes, and verify the deployed role graph before activation.

INFO-02 — Immutable recipients and narrow recovery paths create asset-liveness risk

Status: Open; no remediation retest

Affected code: [`AagentSaleProceedsVault.sol:L30-L93`](#), [`AagentSwapV2.sol:L671-L672`](#), [`AagentRedemptionV2.sol:L895-L897`](#)

The immutable beneficiary can block native release by rejecting BNB. Vault recovery supports only USDT and USDC; forced native currency and unrelated tokens have no recovery path.

Recommendation: Prove beneficiary payable behavior and continuity, then either document intentional irrecoverability or implement constrained/timelocked recovery that excludes protected liabilities.

INFO-03 — Settlement events omit configuration evidence needed for complete reconstruction

Status: Open; no remediation retest

Affected code: [`AagentSwapV2.sol:L81-L90`](#), [`AagentRedemptionV2.sol:L122-L133`](#)

`PurchaseSettled` omits `mode` and `configVersion`; `RedemptionSettled` omits `configVersion`, `rateVersion`, and the cost-basis indicator.

Impact: Settlement correctness is not directly changed, but durable standalone forensic reconstruction is incomplete.

Recommendation: Add the missing fields or a versioned configuration digest and lock the event schema with tests.

11. Positive-control analysis

The following observations support the 0 Critical / 0 High result for the reviewed snapshot:

- purchase and redemption settlement paths are nonreentrant;
- token transfers use `SafeERC20` and reconcile actual balance changes;
- purchase execution is bound to phase, mode, limited-window ID, configuration version, minimum output, and deadline;
- redemption execution is bound to phase, mode, configuration version, rate version, minimum output, deadline, and a per-wallet nonce;
- wallet purchase receipts and cost basis limit presale refunds;
- post-launch redemption collects AGNT;
- claims clear state before transfer;
- eligible withdrawals preserve recorded liabilities and unclaimed claims;
- Solidity 0.8.24 checked arithmetic and `Math.mulDiv` reduce overflow and truncation risk;
- no arbitrary external-call target, `delegatecall`, upgrade proxy, or user-controlled execution hook was observed in scope.

These controls are evidence for the current severity assessment, not a guarantee that unknown vulnerabilities do not exist.

12. Verification evidence

Dynamic tests

- Command: `npm.cmd test`
- Result: **42 passing, 0 failing**
- The run includes the wallet execution-nonce consumption/replay regression.

Coverage

Scope	Statements	Branches	Functions	Lines
All instrumented files	89.32% (301/337)	56.81% (242/426)	77.50% (62/80)	78.67% (472/600)
Four core contracts	89.67% (295/329)	56.40% (238/422)	85.07% (57/67)	79.72% (460/577)
Market Registry	92.45% (49/53)	65.00% (52/80)	100% (12/12)	90.22% (83/92)
Redemption V2	96.20% (152/158)	51.09% (94/184)	92.31% (24/26)	80.59% (220/273)
Sale Proceeds Vault	70.00% (14/20)	50.00% (16/32)	60.00% (3/5)	65.63% (21/32)
Swap V2	81.63% (80/98)	60.32% (76/126)	75.00% (18/24)	75.56% (136/180)

Static analysis

Slither 0.11.5 completed and emitted 55 raw project detector entries: 24 Medium, 27 Low, and 4 Informational. Manual triage did not map raw tool classifications directly to accepted findings.

Detector	Raw count	Threat-model disposition
<code>incorrect-equality</code>	15	Intended exact/zero/enum/sentinel conditions
<code>locked-ether</code>	1	Forced-native and recovery liveness captured by INFO-02
<code>uninitialized-local</code>	7	Zero-initialized locals with guarded assignments
<code>unused-return</code>	1	<code>startedAt</code> unused; round order and <code>updatedAt</code> checked
<code>missing-zero-check</code>	1	Mock-only and outside production scope
<code>reentrancy-events</code>	1	No repeat withdrawal; fixed beneficiary liveness remains
<code>timestamp</code>	25	Intended schedules/deadlines; LOW-01 retained
<code>pragma</code>	1	No separate exploit finding

Detector	Raw count	Threat-model disposition
cyclomatic-complexity	1	Redemption validation maintainability warning
low-level-calls	2	Fixed vault/beneficiary native flows

Toolchain

- Node.js [v24.18.0](#)
- npm [11.16.0](#)
- Hardhat [2.29.0](#)
- ethers [6.17.0](#)
- Solidity [0.8.24+commit.e11b9ed9](#)
- optimizer enabled with 500 runs
- EVM target [paris](#)
- OpenZeppelin Contracts [5.1.0](#)
- solidity-coverage [0.8.17](#)
- Slither [0.11.5](#)
- crytic-compile [0.3.11](#)

Canonical artifact hashes and source/snapshot equality are recorded in [EVIDENCE_MANIFEST.md](#).

13. Remediation and retest strategy

1. Add purchase-time reserve collateral enforcement and three-asset underfunding tests.
2. Add oracle feed registry, per-feed heartbeat, bounds, deviation, stablecoin floor, and automatic pause tests.
3. Make manual review cumulative or reviewer-authorized and test splitting.
4. Replace or formally accept the UTC-bucket burst model.
5. Improve event evidence and decide asset-recovery behavior.
6. Deploy ownership behind a verified multisig/timelock role model.
7. Freeze final source bytes and rerun compilation, all tests, coverage, Slither, manual line review, and source hash comparison.
8. Verify deployed bytecode, constructor arguments, endpoint bindings, token/oracle addresses, roles, pause state, reserve funding, and minimal-wallet transaction receipts.

14. Limitations

- This is a first-party assessment, not independent certification.
- Conclusions apply only to the ten listed source hashes.

- Deployment, bytecode equivalence, live roles, reserve funding, oracle addresses, monitoring, and transaction receipts were not supplied.
- No stateful fuzzing, invariant engine, symbolic execution, formal proof, economic simulation, or operational key-compromise exercise was completed.
- External token and oracle contracts were assumed to conform to their interfaces.
- Static-analysis results are heuristic and require manual interpretation.
- Coverage does not prove correctness and leaves material branch/path gaps.

15. Conclusion

The reviewed source contains meaningful controls against stale configuration, direct replay, reentrancy, double refunds, double claims, liability-violating withdrawals, and unsafe token accounting. The open Medium findings leave material gaps in reserve assurance, oracle integrity, and manual-review enforcement.

Final disposition: remediation required before mainnet.